Protección de rutas y recursos por rol

Matías Lira Breyan Sebastián

—

Desarrollo Web Profesional

—

José Miguel Carrera Pacheco

¿QUÉ ES UN MIDDLEWARE?

Un middleware es una función o componente que se ejecuta entre la solicitud del usuario (request) y la respuesta del servidor (response).

Su función principal es interceptar, analizar o modificar la petición antes de que llegue al destino final.

¿Para qué sirve?

El middleware se utiliza para:

- verificar autenticación
- validar permisos
- registrar actividad (logs)
- manejar errores
- procesar datos

Ejemplo conceptual

Flujo de una petición:

Usuario → Middleware → Servidor → Respuesta

Ejemplo en JavaScript (estilo Express):

```
function verificarAuth(req, res, next){
    if(req.usuario){
        next(); // continuar
    } else {
        res.send("Acceso denegado");
    }
}
```

¿CÓMO SE PROTEGEN RUTAS EN BACKEND?

Proteger rutas significa evitar que usuarios no autorizados accedan a ciertas partes del sistema.

Uso de middleware de autenticación

Se utiliza un middleware para verificar si el usuario ha iniciado sesión.

Ejemplo:

```
app.get("/dashboard", verificarAuth, function(req, res){
    res.send("Bienvenido");
});
```

Si el usuario no está autenticado, no podrá acceder a la ruta.

Validación de tokens o sesiones

El backend puede verificar:

- tokens (como JWT)
- sesiones activas

Esto asegura que la solicitud proviene de un usuario válido.

Protección por tipo de usuario

No todos los usuarios deben acceder a lo mismo, por lo que se combinan autenticación y autorización.

¿QUÉ ERRORES DE SEGURIDAD SON COMUNES?

Existen errores frecuentes que pueden comprometer la seguridad de una aplicación web.
Muchos de ellos son documentados por la OWASP Foundation.

Falta de validación de autenticación

Permitir acceso a rutas sin verificar si el usuario inició sesión.

Exposición de datos sensibles

Mostrar información como:

- contraseñas
- tokens
- datos personales

Validación solo en frontend

Confiar únicamente en validaciones del lado del cliente es un error, ya que pueden ser manipuladas.

Inyección de código

No validar correctamente entradas puede permitir ataques como:

- SQL Injection
- ejecución de scripts maliciosos

Manejo incorrecto de errores

Mostrar errores técnicos al usuario puede revelar información del sistema.

¿CÓMO SE VALIDA ACCESO POR ROL?

La validación por rol consiste en permitir o negar acceso según el tipo de usuario.

Este modelo se basa en el control de acceso por roles (RBAC).

Ejemplo de roles

- administrador
- usuario
- invitado

Ejemplo de middleware por rol

```
function verificarRol(rol){
    return function(req, res, next){
        if(req.usuario.rol === rol){
            next();
        } else {
            res.send("No tienes permiso");
        }
    }
}
```

Uso:

```
app.get("/admin", verificarAuth, verificarRol("admin"),
function(req, res){
    res.send("Panel de administrador");
});
```

Buenas prácticas

- verificar roles en el backend
- no confiar en datos del frontend
- usar bases de datos para definir roles y permisos
- aplicar el principio de mínimos privilegios (dar solo los permisos necesarios)